

EJERCICIOS BASE DE DATOS RELACIONES CON JAVA

0) Preparación en MySQL (Workbench o consola)

✓ Enunciado 0

1. Crea una base de datos llamada **empresa1**.
2. Crea la tabla **clientes** con los campos: id, nif, nombre, edad.
3. Inserta 3 registros de prueba.

✓ SQL (MySQL)

```
CREATE DATABASE IF NOT EXISTS empresa1;
USE empresa1;

DROP TABLE IF EXISTS clientes;

CREATE TABLE clientes (
  id INT PRIMARY KEY,
  nif VARCHAR(10) NOT NULL,
  nombre VARCHAR(60) NOT NULL,
  edad VARCHAR(2)
);

INSERT INTO clientes (id, nif, nombre, edad) VALUES
(1, '11111', 'Alberto', '40'),
(2, '22222', 'Ana', '20'),
(3, '33333', 'Luis', '22');
```

Clase común (recomendable): DB.java

Para no repetir URL/usuario/clave en cada ejercicio (muy cómodo en clase). Esto encaja con la idea de “helper” de la unidad

unidad9

.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
```

```
public class DB {

    private static final String URL = "jdbc:mysql://localhost/empresa1";
    private static final String USER = "root";
    private static final String PASS = "";

    public static Connection conectar() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

✓ EJERCICIO 1 (Java + Statement): Crear la BD empresa1

Enunciado: desde Java crea la base de datos empresa1 usando Statement.
(Basado en tu CrearBaseDeDatos.java)

```
CrearBaseDeDatos
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class Ej1_CrearBaseDeDatos {
    public static void main(String[] args) {

        String sql = "CREATE DATABASE IF NOT EXISTS empresa1";

        try (Connection con = DriverManager.getConnection("jdbc:mysql://localhost/",
"root", ""));
            Statement st = con.createStatement()) {

            st.execute(sql);
            System.out.println("BD empresa1 creada/ya existía.");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

✓ EJERCICIO 2 (Java + Statement): Crear tabla clientes

Enunciado: crea la tabla clientes.
(Basado en tu CrearTabla.java)

```
CrearTabla
import java.sql.Connection;
```

```

import java.sql.Statement;

public class Ej2_CrearTablaClientes {
    public static void main(String[] args) {

        String sql = "CREATE TABLE IF NOT EXISTS clientes (" +
            "id INT PRIMARY KEY," +
            "nif VARCHAR(10) NOT NULL," +
            "nombre VARCHAR(60) NOT NULL," +
            "edad VARCHAR(2))";

        try (Connection con = DB.conectar();
            Statement st = con.createStatement()) {

            st.execute(sql);
            System.out.println("Tabla clientes creada/ya existía.");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

✓ EJERCICIO 3 (Java + Statement): Insertar un registro fijo

Enunciado: inserta un cliente fijo con Statement.

(Basado en tu InsertarDatos.java)

```

InsertarDatos
import java.sql.Connection;
import java.sql.Statement;

public class Ej3_InsertarFijoStatement {
    public static void main(String[] args) {

        String sql = "INSERT INTO clientes (id, nif, nombre, edad) " +
            "VALUES (10,'99999','ClienteFijo','30')";

        try (Connection con = DB.conectar();
            Statement st = con.createStatement()) {

            int filas = st.executeUpdate(sql);
            System.out.println("Insertados: " + filas);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

```
}  
}
```

✓ EJERCICIO 4 (Java + PreparedStatement): Insertar pidiendo datos

Enunciado: pide por teclado id, nif, nombre, edad e inserta con PreparedStatement (parámetros ?).
(Esto es lo que la unidad recomienda para parámetros)

```
import java.sql.Connection;  
import java.sql.PreparedStatement;  
import java.util.Scanner;  
  
public class Ej4_InsertarPrepared {  
    public static void main(String[] args) {  
  
        try (Scanner sc = new Scanner(System.in);  
             Connection con = DB.conectar()) {  
  
            System.out.print("ID: ");  
            int id = Integer.parseInt(sc.nextLine());  
  
            System.out.print("NIF: ");  
            String nif = sc.nextLine();  
  
            System.out.print("Nombre: ");  
            String nombre = sc.nextLine();  
  
            System.out.print("Edad: ");  
            String edad = sc.nextLine();  
  
            String sql = "INSERT INTO clientes (id, nif, nombre, edad) VALUES (?, ?, ?, ?)";  
  
            try (PreparedStatement ps = con.prepareStatement(sql)) {  
                ps.setInt(1, id);  
                ps.setString(2, nif);  
                ps.setString(3, nombre);  
                ps.setString(4, edad);  
  
                int filas = ps.executeUpdate();  
                System.out.println("Insertados: " + filas);  
            }  
  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```
}
```

✓ EJERCICIO 5 (Java + Statement): Listar todos (SELECT)

Enunciado: muestra todos los clientes por pantalla.

(Basado en tu MostrarDatos.java)

MostrarDatos

```
import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.Statement;

public class Ej5_ListarClientes {
    public static void main(String[] args) {

        String sql = "SELECT * FROM clientes";

        try (Connection con = DB.conectar();
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery(sql)) {

            while (rs.next()) {
                System.out.println(
                    rs.getInt("id") + " | " +
                    rs.getString("nif") + " | " +
                    rs.getString("nombre") + " | " +
                    rs.getString("edad")
                );
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

✓ EJERCICIO 6 (Java + PreparedStatement): Update y Delete (2 en 1)

Enunciado:

A) Actualiza la edad de un cliente por id (UPDATE).

B) Borra un cliente por id (DELETE).

(La unidad explica UPDATE/DELETE con executeUpdate() y parámetros)

```
import java.sql.Connection;
import java.sql.PreparedStatement;
```

```

import java.util.Scanner;

public class Ej6_UpdateYDeletePrepared {
    public static void main(String[] args) {

        try (Scanner sc = new Scanner(System.in);
            Connection con = DB.conectar()) {

            // A) UPDATE
            System.out.print("ID a actualizar: ");
            int idUpdate = Integer.parseInt(sc.nextLine());

            System.out.print("Nueva edad: ");
            String nuevaEdad = sc.nextLine();

            String sqlUpdate = "UPDATE clientes SET edad = ? WHERE id = ?";
            try (PreparedStatement ps = con.prepareStatement(sqlUpdate)) {
                ps.setString(1, nuevaEdad);
                ps.setInt(2, idUpdate);
                System.out.println("Filas actualizadas: " + ps.executeUpdate());
            }

            // B) DELETE
            System.out.print("ID a borrar: ");
            int idDelete = Integer.parseInt(sc.nextLine());

            String sqlDelete = "DELETE FROM clientes WHERE id = ?";
            try (PreparedStatement ps = con.prepareStatement(sqlDelete)) {
                ps.setInt(1, idDelete);
                System.out.println("Filas borradas: " + ps.executeUpdate());
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

RETO

Perfecto Alberto 🐼

Te preparo ahora **una práctica completa nueva**, pero creando **otra base de datos distinta**, lista para entregar a los alumnos.

PRÁCTICA COMPLETA – JDBC CON OTRA BASE DE DATOS

Objetivo

Trabajar:

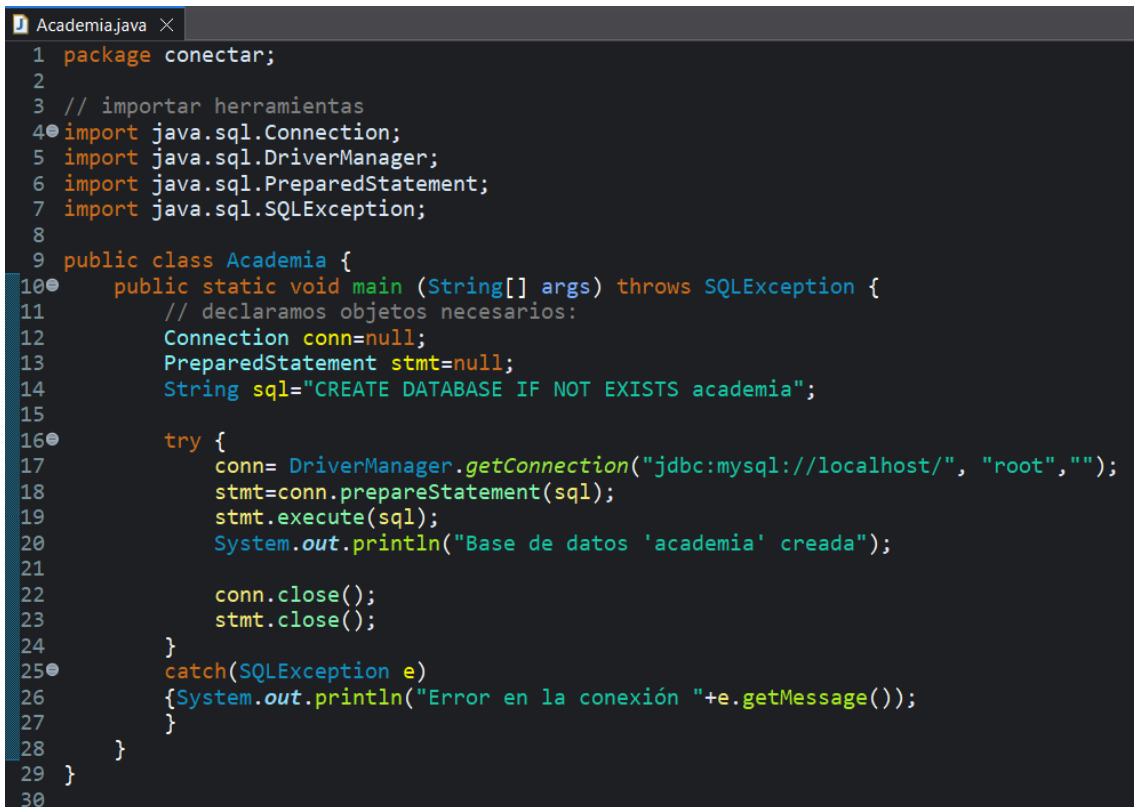
- Conexión JDBC
- PreparedStatement
- SELECT, INSERT, UPDATE, DELETE
- ResultSet

PARTE 1 – CREACIÓN DE BASE DE DATOS (MySQL)

Enunciado 0

1. Crear una base de datos llamada:

Academia

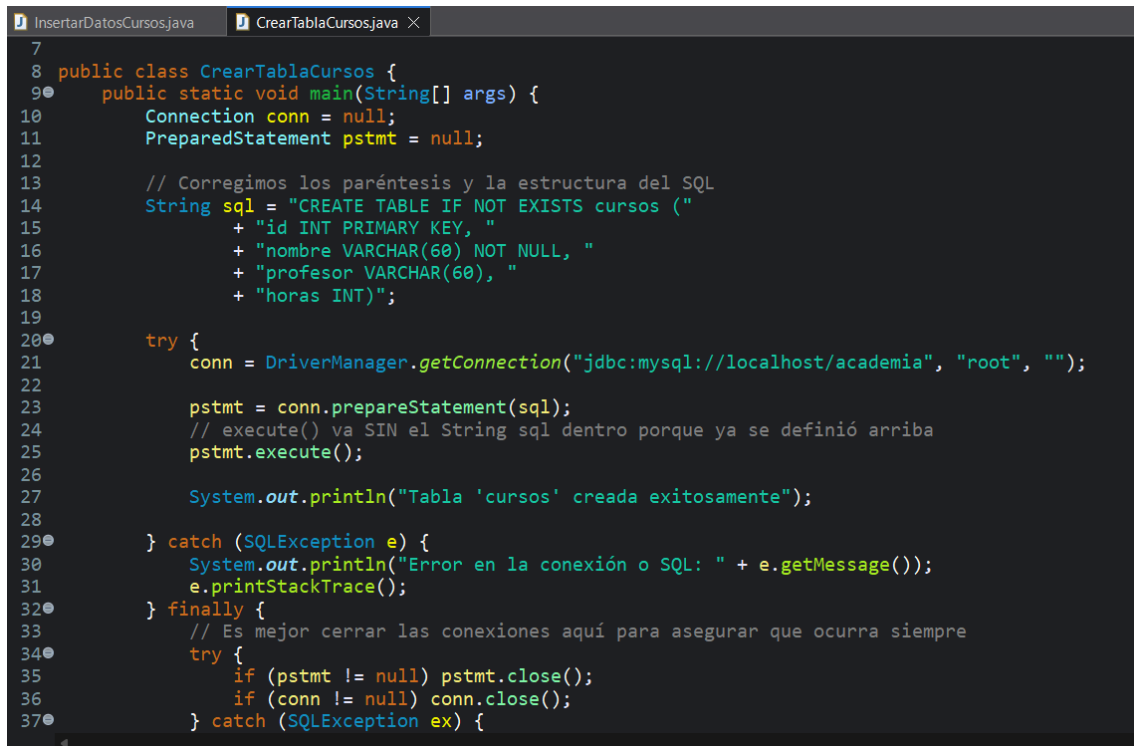


```
1 package conectar;
2
3 // importar herramientas
4 import java.sql.Connection;
5 import java.sql.DriverManager;
6 import java.sql.PreparedStatement;
7 import java.sql.SQLException;
8
9 public class Academia {
10     public static void main (String[] args) throws SQLException {
11         // declaramos objetos necesarios:
12         Connection conn=null;
13         PreparedStatement stmt=null;
14         String sql="CREATE DATABASE IF NOT EXISTS academia";
15
16         try {
17             conn= DriverManager.getConnection("jdbc:mysql://localhost/", "root","");
18             stmt=conn.prepareStatement(sql);
19             stmt.execute(sql);
20             System.out.println("Base de datos 'academia' creada");
21
22             conn.close();
23             stmt.close();
24         }
25         catch(SQLException e)
26         {System.out.println("Error en la conexión "+e.getMessage());
27         }
28     }
29 }
30
```

2. Crear una tabla llamada: cursos

Con los siguientes campos:

- id (INT, clave primaria)
- nombre (VARCHAR 50)
- profesor (VARCHAR 50)
- horas (INT)

A screenshot of a Java IDE with two tabs: 'InsertarDatosCursos.java' and 'CrearTablaCursos.java'. The 'CrearTablaCursos.java' tab is active, showing a Java class with a main method. The code creates a MySQL connection, prepares an SQL statement to create a table named 'cursos' with columns 'id' (INT PRIMARY KEY), 'nombre' (VARCHAR(60) NOT NULL), 'profesor' (VARCHAR(60)), and 'horas' (INT). It then executes the statement and prints a success message. Error handling is included with try-catch-finally blocks to close resources.

```
7
8 public class CrearTablaCursos {
9     public static void main(String[] args) {
10         Connection conn = null;
11         PreparedStatement pstmt = null;
12
13         // Corregimos los paréntesis y la estructura del SQL
14         String sql = "CREATE TABLE IF NOT EXISTS cursos ("
15             + "id INT PRIMARY KEY, "
16             + "nombre VARCHAR(60) NOT NULL, "
17             + "profesor VARCHAR(60), "
18             + "horas INT)";
19
20     try {
21         conn = DriverManager.getConnection("jdbc:mysql://localhost/academia", "root", "");
22
23         pstmt = conn.prepareStatement(sql);
24         // execute() va SIN el String sql dentro porque ya se definió arriba
25         pstmt.execute();
26
27         System.out.println("Tabla 'cursos' creada exitosamente");
28
29     } catch (SQLException e) {
30         System.out.println("Error en la conexión o SQL: " + e.getMessage());
31         e.printStackTrace();
32     } finally {
33         // Es mejor cerrar las conexiones aquí para asegurar que ocurra siempre
34     try {
35         if (pstmt != null) pstmt.close();
36         if (conn != null) conn.close();
37     } catch (SQLException ex) {
```

PARTE 2 – EJERCICIOS EN JAVA

Base de datos: academia

Tabla: cursos


```

1 package conectar;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.PreparedStatement;
6 import java.sql.SQLException;
7
8 public class InsertarDatosCursos {
9     public static void main(String[] args) {
10         String sql = "INSERT INTO cursos (id, nombre, profesor, horas) " +
11             "VALUES (1, 'Programación Sistemas', 'Alberto', '520'), "
12             + "(2, 'POO', 'Alberto', '500'), "
13             + "(3, 'Seguridad Informática', 'Nacho', '350')";
14
15         // 2. Usar try-with-resources para auto-cerrar conexión y statement
16         try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost/academia", "root", "");
17             PreparedStatement stmt = conn.prepareStatement(sql)) {
18
19             int filasAfecadas = stmt.executeUpdate();
20             System.out.println("Datos insertados. Filas afectadas: " + filasAfecadas);
21
22         } catch (SQLException e) {
23             System.out.println("Error en la conexión o SQL: " + e.getMessage());
24             e.printStackTrace();
25         }
26     }
27 }
28

```

● EJERCICIO 1 – Insertar curso con PreparedStatement

📌 Enunciado

Crear un programa que:

1. Se conecte a la base de datos academia.
2. Inserte un curso usando PreparedStatement.
3. Muestre el número de filas insertadas.

El curso puede estar definido directamente en el código.

(CAPTURAS ARRIBA)

● EJERCICIO 2 – Insertar curso con PreparedStatement

📌 Enunciado

Crear un programa que:

1. Pida por teclado:
 - id
 - nombre
 - profesor
 - horas

2. Inserte el curso usando PreparedStatement.
3. Muestre el número de filas insertadas.

(CAPTURAS ARRIBA)

● EJERCICIO 3 – Mostrar todos los cursos

📌 Enunciado

Crear un programa que:

1. Haga un SELECT * FROM cursos.
2. Recorra el ResultSet.
3. Muestre todos los cursos en formato:

ID | NOMBRE | PROFESOR | HORAS

```
InsertarDatosCursos.java  CrearTablaCursos.java  MostrarCursos.java X
4 public class MostrarCursos {
5
6     // Configuración de la conexión (Ajusta según tu base de datos)
7     private static final String URL = "jdbc:mysql://localhost:3306/academia";
8     private static final String USER = "root";
9     private static final String PASS = "";
10
11     public static void main(String[] args) {
12
13         // 1. Definir la consulta SQL
14         String sql = "SELECT * FROM cursos";
15
16         // 2. Usar Try-with-resources para asegurar el cierre de la conexión automáticamente
17         try (Connection con = DriverManager.getConnection(URL, USER, PASS);
18             PreparedStatement st = con.prepareStatement(sql);
19             ResultSet rs = st.executeQuery(sql)) {
20
21             System.out.println("LISTADO DE CURSOS");
22
23             // 3. Recorrer el ResultSet (Iteración sobre las filas)
24             while (rs.next()) {
25                 // Obtención de datos por nombre de columna
26                 int id = rs.getInt("id");
27                 String nombre = rs.getString("nombre");
28                 String profesor = rs.getString("profesor");
29                 int horas = rs.getInt("horas");
30
31                 // 4. Mostrar en el formato solicitado
32                 System.out.println("id: "+id+"\nnombre: "+nombre+"\nprofesor: "+profesor+"\nhoras: "+horas);
33             }
34         }
35     }
36 }
```

● EJERCICIO 4 – Buscar curso por nombre

📌 Enunciado

Crear un programa que:

1. Pida un nombre de curso.
2. Busque el curso usando PreparedStatement.

3. Si existe, mostrar sus datos.
4. Si no existe, mostrar mensaje correspondiente.

Ejemplo de sentencia

Update productos set precio=? Where id=?

Temenos que usar dos set

```
InsertarDatosCursos.java  CrearTablaCursos.java  MostrarCursos.java  *BuscarCursoNombre.java X
7  private static final String URL = "jdbc:mysql://localhost:3306/academia";
8  private static final String USER = "root";
9  private static final String PASS = "";
10
11  public static void main(String[] args) {
12      Scanner sc = new Scanner(System.in);
13
14      System.out.print("Introduce el nombre del curso a buscar: ");
15      String nombreBuscar = sc.nextLine();
16
17      // marcador de posición '?'
18      String sql = "SELECT * FROM cursos WHERE nombre = ?";
19
20      try (Connection con = DriverManager.getConnection(URL, USER, PASS);
21          PreparedStatement ps = con.prepareStatement(sql)) {
22
23          // rellenamos el marcador (índice 1)
24          ps.setString(1, nombreBuscar);
25
26          // ejecutamos la consulta
27          try (ResultSet rs = ps.executeQuery()) {
28
29              // comprobamos resultados
30              if (rs.next()) {
31                  System.out.println("\n--- CURSO ENCONTRADO ---");
32                  System.out.println("ID: " + rs.getInt("id"));
33                  System.out.println("NOMBRE: " + rs.getString("nombre"));
34                  System.out.println("PROFESOR: " + rs.getString("profesor"));
35                  System.out.println("HORAS: " + rs.getInt("horas"));
36              }
37          }
38      }
39  }
```

EJERCICIO 5 – Actualizar horas de un curso

Enunciado

Crear un programa que:

1. Pida un ID.
2. Pida el nuevo número de horas.
3. Actualice el curso correspondiente.
4. Muestre el número de filas modificadas.

● EJERCICIO 6 – Eliminar curso

🚩 Enunciado

Crear un programa que:

1. Pida un ID.
2. Elimine el curso correspondiente.
3. Muestre el número de filas eliminadas.

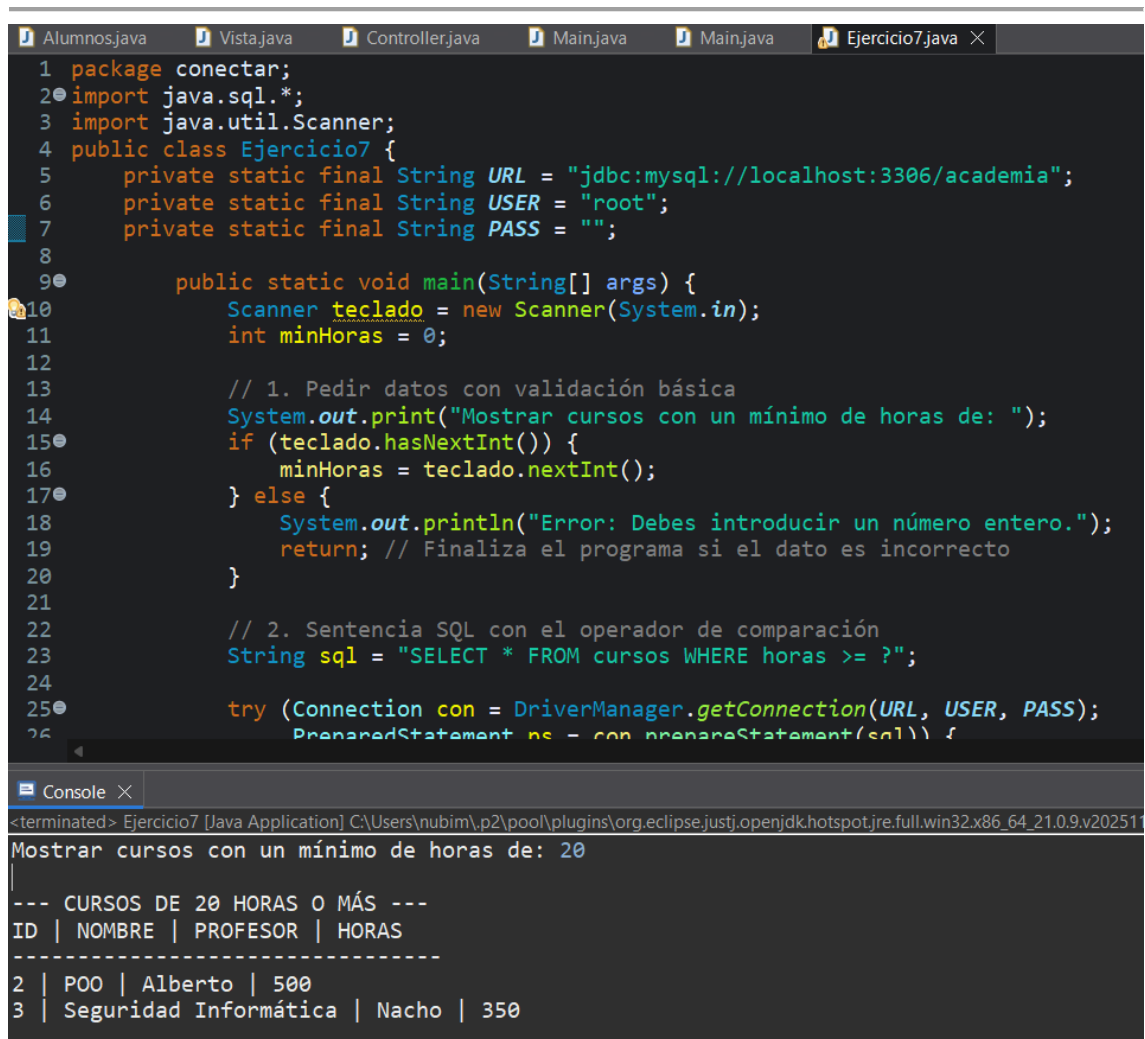
```
EliminarCurso.java X
1 import java.sql.*;
2 import java.util.Scanner;
3 public class EliminarCurso {
4
5
6
7     private static final String URL = "jdbc:mysql://localhost:3306/academia";
8     private static final String USER = "root";
9     private static final String PASS = "";
10
11     public static void main(String[] args) {
12         Scanner sc = new Scanner(System.in);
13
14         // 1. Pedir el ID al usuario
15         System.out.print("Introduce el ID del curso que deseas eliminar: ");
16         int idEliminar = sc.nextInt();
17
18         // 2. Definir la sentencia SQL con parámetro
19         String sql = "DELETE FROM cursos WHERE id = ?";
20
21         try (Connection con = DriverManager.getConnection(URL, USER, PASS);
22             PreparedStatement ps = con.prepareStatement(sql)) {
23
24             // 3. Asignar el ID al marcador '?'
25             ps.setInt(1, idEliminar);
26
27             // 4. Ejecutar la eliminación y capturar las filas afectadas
28             int filasAfectadas = ps.executeUpdate();
29
30             // 5. Mostrar resultado según el número de filas eliminadas
31             if (filasAfectadas > 0) {
4.
Introduce el ID del curso que deseas eliminar: 1
Éxito: Se ha eliminado el curso con ID 1
Total de filas eliminadas: 1
```

● EJERCICIO 7

🚩 Enunciado

Crear un programa que:

1. Pida un número mínimo de horas.
2. Muestre todos los cursos con horas mayores o iguales a ese número.
3. Use PreparedStatement.



```
1 package conectar;
2 import java.sql.*;
3 import java.util.Scanner;
4 public class Ejercicio7 {
5     private static final String URL = "jdbc:mysql://localhost:3306/academia";
6     private static final String USER = "root";
7     private static final String PASS = "";
8
9     public static void main(String[] args) {
10         Scanner teclado = new Scanner(System.in);
11         int minHoras = 0;
12
13         // 1. Pedir datos con validación básica
14         System.out.print("Mostrar cursos con un mínimo de horas de: ");
15         if (teclado.hasNextInt()) {
16             minHoras = teclado.nextInt();
17         } else {
18             System.out.println("Error: Debes introducir un número entero.");
19             return; // Finaliza el programa si el dato es incorrecto
20         }
21
22         // 2. Sentencia SQL con el operador de comparación
23         String sql = "SELECT * FROM cursos WHERE horas >= ?";
24
25         try (Connection con = DriverManager.getConnection(URL, USER, PASS);
26             PreparedStatement ps = con.prepareStatement(sql)) {
```

Console

<terminated> Ejercicio7 [Java Application] C:\Users\nubim\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v202511

Mostrar cursos con un mínimo de horas de: 20

ID	NOMBRE	PROFESOR	HORAS
2	POO	Alberto	500
3	Seguridad Informática	Nacho	350

EJERCICIO FINAL

Crear un único programa con menú:

1. Insertar curso
2. Mostrar cursos
3. Buscar curso
4. Actualizar curso
5. Eliminar curso
0. Salir

```
Alumnos.java Vista.java Controller.java Main.java Main.java EjercicioFinal.java X
1 package conectar;
2 import java.sql.*;
3 import java.util.Scanner;
4 public class EjercicioFinal {
5
6
7     // conexión
8     private static final String URL = "jdbc:mysql://localhost:3306/academia";
9     private static final String USER = "root";
10    private static final String PASS = "";
11    private static Scanner teclado = new Scanner(System.in);
12
13    public static void main(String[] args) {
14        int opcion = -1;
15
16        do {
17            System.out.println("\n--- MENÚ GESTIÓN ACADEMIA ---");
18            System.out.println("1. Insertar curso");
19            System.out.println("2. Mostrar todos los cursos");
20            System.out.println("3. Buscar curso por nombre");
21            System.out.println("4. Actualizar horas de un curso");
22            System.out.println("5. Eliminar curso");
23            System.out.println("0. Salir");
24            System.out.print("Selecciona una opción: ");
25
13
14
15
16
17
18
19
20
21
22
23
24
25

```

EjercicioFinal [Java Application] C:\Users\nubim\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v20251105-0741\jre\bin\javaw.exe

```
--- MENÚ GESTIÓN ACADEMIA ---
1. Insertar curso
2. Mostrar todos los cursos
3. Buscar curso por nombre
4. Actualizar horas de un curso
5. Eliminar curso
0. Salir
Selecciona una opción:

```